AI Review Responder Workspace

PROJECT ARCHITECTURE & LIFECYCLE DOCUMENTATION (V1.0 BASELINE)

1. Executive Summary & Project Description

The **AI Review Responder Workspace** is a smart, localized operational dashboard designed to ingest, monitor, evaluate, and respond to incoming customer reviews. Built primarily as a highly responsive Minimum Viable Product (MVP), the system automates brand-compliant drafting workflows while acting as a vigilant real-time defensive shield against high-risk business reputation crises.

By leveraging generative AI models alongside local transaction logging frameworks and external warning channels, the system allows business owners to instantly craft tailored, intelligent replies that respect strict brand configurations, tone restraints, and specific contextual nuances.

Core Technical Stack Base

Streamlit UI | Google Gemini API | SendGrid Email Engine | SQLite3 Transactional Ledger | Git & GitHub

2. Project Architecture Blueprint

The platform utilizes a structured modular setup to prevent cross-contamination of logic and ensure strict boundaries between the user interface layout and the core logical engines:

- **Frontend Framework (`app.py`):** Implements a localized stateful UI featuring isolated rendering fragments to execute automated ingestion simulators asynchronously without stalling the parent interface container canvas.
- **AI Core Module (`responder_engine.py` & `analytics_engine.py`):** Interfaces directly with Gemini API to run prompt injections contextually bounded by brand instruction rules.
- **Database Warehouse (`database_manager.py`):** Maintains an offline transactional SQLite ledger to capture customer inputs, operational metadata logs, approval flags, and dispatch statuses.
- **Notification System (`notifier_engine.py`):** Integrates SendGrid APIs to isolate high-risk feedback loops and escalate emergency emails directly to corporate stakeholders.

3. Phase-by-Phase Development Lifecycle

The construction of this MVP was systematically executed across distinct developmental sprints, ensuring that each piece of infrastructure was validated and hardened before advancing:

Phase 1: Foundation Setup & Ingestion Simulation

Established the fundamental workspace environment layout and local mock JSON datasets. Engineered an ingestion simulator to mimic continuous background real-time api data streams, supplying the workspace text fields with structured, multi-tier review scenarios.

Phase 2: Database Layer & State Hardening

Constructed the relational SQLite schema to prevent text field tracking overlaps. Unified Streamlit's operational session state dictionaries directly with database row generation to make the text inputs highly persistent across UI rerun cycles.

Phase 3: AI Inference & Brand Compliance Injection

Configured the primary analytical pipelines. Designed system context injections to read custom brand profile parameters (e.g., `brand_rules.json`), producing responses optimized for specific localized tones while dynamically tracking metric properties like response generation latency (T_{gen}).

Phase 4: Real-Time Escalation Shield & Notification Routing

Implemented an automated risk-scanning matrix. If incoming customer review data contains critical escalations (such as legal threats or severe brand hazards), the system instantly forces an emergency breakout loop. It flashes an operational alert box on the screen and concurrently dispatches an urgent email update via SendGrid before any manual approval action occurs.

Phase 5: Fragment Synchronization & Thread Debugging

Refactored the workspace rendering container using Streamlit's localized `@st.fragment` wrappers to handle automated background polling cycles seamlessly. Eliminated architectural UI errors by abstracting component state assignments to dedicated global tracking parameters, shielding outer layout boundaries from fragment updates.

Phase 6: Remote Cloud Versioning Control Baseline

Configured a clean repository directory layout. Built a robust `.gitignore` structure to permanently block development environmental variables (`.env`), internal execution artifacts (`__pycache__/`), and localized transaction databases from public cloud visibility. Successfully deployed the clean MVP baseline code live to GitHub.

4. Core Project Directory Structure

File / Directory Pathway	Primary Architectural Responsibility
`app.py`	Main dashboard layout, session state coordination, and fragment rendering bounds.
`responder_engine.py`	Manages AI text generation, prompt compilation, and brand rule compliance.
`analytics_engine.py`	Handles sentiment breakdown classification and review analytics tracking.
`database_manager.py`	Performs local SQLite query operations, transaction records logging, and data persistence.
`notifier_engine.py`	Controls high-risk escalation detection and SendGrid email dispatch routing loops.
`ingestion_simulator.py`	Simulates real-time automated streaming review inputs for sandbox testing.
`config/`	Contains brand configuration rules, boutique guidelines, and mock review datasets.
`.gitignore`	Defends the cloud repository by excluding local credentials, virtual environments, and caches.

5. Future Technical Roadmap Recommendations

Next Milestone Focus: As the system moves from an interactive MVP to a production architecture, it is recommended to transition from the current *Synchronous Dashboard-Driven Architecture* to an *Asynchronous Event-Driven Architecture*. This involves decoupling the ingestion pipeline into a standalone cloud worker process powered by database messaging queues, ensuring that 24/7 high-risk escalation alerts occur immediately even when the human monitoring interface is completely closed.